Assignment 3

Control Allocation

Complete the provided MATLAB function to convert control-group inputs into individual control-effector commands for the Hero lift-plus-cruise vehicle. Use the supplied caller script to exercise 3–5 input cases and display a table with one column per case. First run the provided P-code solution to check the caller and see the expected results, then implement and test your own function. This function will support the vehicle trim and linearization work that follows.

Figure 1 identifies the nine effectors using the same numbering as the control allocation function.

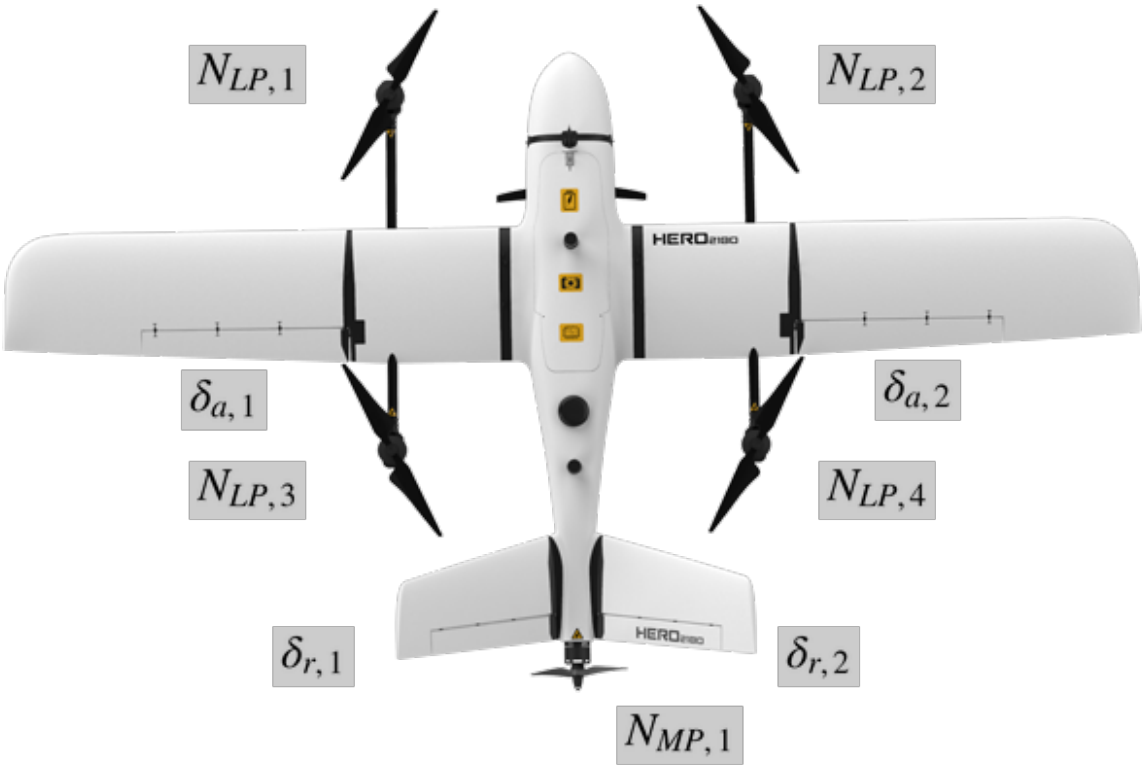


Figure 1: Hero VTOL control effectors, viewed from above. $\delta_{a,i}$ denotes an aileron, $\delta_{r,i}$ a ruddervator, and N a propeller RPM command. Numbering follows `ControlAllocation.m`. Base image: Makeflyeasy.

Function interface

Save your completed template as `ControlAllocation.m` and retain this interface:

```
U = ControlAllocation(UTrimVec,FltCon,Veh);
```

`UTrimVec` contains the following control groups. Use positive surface deflection for trailing-edge down. Body axes follow the right-handed forward–right–down convention.

Index	Input	Definition
1–3	`ulat, ulon, udir`	Lateral, longitudinal, and directional commands; each limited to $[-1, +1]$.
4	`MPRPM`	Common cruise-propeller command (RPM).
5	`LPRPM`	Common lift-propeller command (RPM).
6	`NAC`	Nacelle angle (deg); retained in the interface.
7 (optional)	`dflp`	Direct flap command (deg). Omit this element to use the provided automatic schedule.

Return U as a 9×1 **column vector** in this exact order:

Index	Effector	Units / order
1–2	Outboard ailerons	Left, then right (deg).
3–4	Ruddervators	Left, then right (deg).
5–8	Lift propellers	LP-1, LP-2, LP-3, LP-4 (RPM).
9	Cruise propeller	MP-1 (RPM).

Vehicle and flight-condition inputs

The supplied caller defines `FltCon.KEAS = 0.01` and `Veh.Aero.VTXN = 30`, both in knots. The differential RPM scales are already supplied inside the function: `dN_lat = 1500`, `dN_lon = 1500`, and `dN_dir = 2000` RPM. Keep these values as provided. Complete `dBmat_LP` directly inside the function; the test script does not need to define `Veh.ControlAllocation` fields.

Scope of this version

The supplied function has nine effectors. `NAC` and `dflp` are retained for compatibility but do not change U in this version. Keep all sections marked **DO NOT EDIT**; do not add nacelle or flap outputs.

Run the reference solution first

Place the provided `ControlAllocation.p` in the current MATLAB folder and run `Control_Test_Script.m`. Confirm that the printed allocator path ends in `.p`, and record the reference table before completing your template.
To test your own `ControlAllocation.m`, move the P-code file outside both the current folder and the MATLAB path. Rerun the caller with identical inputs and confirm that the printed path ends in `.m`. The caller clears the loaded function and refreshes lookup each run. MATLAB gives a same-named `.p` file precedence over `.m` in the same folder (MathWorks documentation).

Complete and Exercise the Function

Part 1: Complete the student template

Complete only the blank entries in the two allocation sections:

1. **Control Effector Allocation.** Replace the empty brackets in `da1`, `da2`, `dr1`, and `dr2` with the appropriate expressions. Use the normalized control inputs and supplied maximum deflections. The ailerons respond to lateral input, while each ruddervator combines longitudinal and directional inputs. Determine the signs using the surface conventions in the template. The ruddervator limits are already implemented.
2. **Propulsor Allocation.** Fill all 16 entries of the 4×4 matrix `dBmat_LP`. Replace each complete `+[]` placeholder with either $+1$ or -1. Rows correspond to LP-1 through LP-4; columns correspond to collective, roll, pitch, and yaw. Use the propeller locations and rotation directions listed beside the matrix rows. For yaw, consider the reaction torque on the aircraft.

Input unpacking and limiting, scheduling, differential RPM calculations, matrix multiplication, and assembly of U are already provided. The differential RPM commands already include the airspeed wash-out weights. Retain this code and concentrate on the missing surface expressions and matrix signs.

Part 2: Exercise `Control_Test_Script.m`

Use or adapt the supplied caller for **3–5 cases** with different combinations of control-group inputs. The five provided cases are neutral, full positive roll, full positive pitch, full positive yaw, and combined half-positive inputs, all with common commands of 5000 RPM for the lift propellers and 2000 RPM for the cruise propeller. If you change the cases, retain a neutral case, an isolated-input case, and a combined-input case, and exercise all three axes across the set.

Use an airspeed at or below $V_{\mathrm{TXN}} - 5$ knots so the differential lift-propeller commands remain active. Choose common RPM values that keep the mixed commands within the vehicle's permitted range; the mixer does not impose RPM limits. Separate each returned U into its nine individual effector commands.

Results table

Have your script create and display **one MATLAB table with a separate column for each case**. Name the case columns `Case1`, `Case2`, and so on, for a total of 3–5 case columns. Use row names to identify the quantities and their units.

- **Input rows:** airspeed, `ulat`, `ulon`, `udir`, `MPRPM`, `LPRPM`, and `NAC`. Include `dflp` if you use the seventh input.
- **Output rows:** one row for each of the nine entries of U, in the documented order: left/right ailerons, left/right ruddervators, LP-1 through LP-4, and MP-1.

Each case column must contain that case's inputs and calculated outputs so the allocation can be checked directly. Label surface deflections in degrees, propeller commands in RPM, airspeed in knots, and normalized inputs as dimensionless. Confirm that each call returns nine finite values in a column vector. Compare your `.m` results with the P-code reference for identical inputs; all nine effector commands should agree.

Submit two MATLAB files

Submit `ControlAllocation.m` and `Control_Test_Script.m` as separate files, **not in a ZIP**. The function must be standalone, rather than embedded in the script. Include your name in both file headers and use MATLAB R2024a-compatible syntax. The caller must reproduce your table using your own `.m` implementation. Do not submit the provided `.p` file. No plots or separate written report are required.